

Recolección de Textos de Internet y Procesamiento

Minería de Textos: curso 25/26

Miquel Esplà Gomis

*Departament de Llenguatges i Sistemes Informàtics
Universitat d'Alacant*

26 de marzo de 2026

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

Recolección de textos de internet y minería de textos

- Recolección automatizada de grandes volúmenes de datos textuales.
- Análisis de tendencias y extracción de conocimiento a partir de datos web.
- Esenciales para entrenar modelos de PLN con datos actualizados y relevantes.
- Aplicaciones en investigación, inteligencia de negocios y análisis de opinión.

¿Qué es el Web Scrapping?

- Técnica para extraer información de páginas web de forma automatizada.
- Se obtiene el contenido HTML y se procesa para extraer datos específicos.
- Alternativas: páginas web estáticas (scrapping tradicional) dinámicas con JavaScript (scrapping con navegadores headless)

Aplicaciones del Web Scraping

- Extracción de datos para análisis (ej. precios de productos, opiniones, noticias).
- Creación de bases de datos a partir de sitios web.
- Monitoreo de cambios en sitios web.

¿Qué es el Web Crawling?

- Proceso de navegación automatizada a través de enlaces web.
- Utilizado por motores de búsqueda para indexar páginas.
- Puede incluir técnicas de web scraping para extraer información.
- Herramientas populares: Scrapy, Heritrix, Nutch.

Aplicaciones del Web Crawling

- Indexación de contenido para motores de búsqueda.
- Recolección de grandes volúmenes de texto para PLN.
- Creación de copias de seguridad de sitios web.

Algunos casos interesantes de crawling

- Internet Archive: <https://archive.org/>
- CommonCrawl: <https://commoncrawl.org/>
- OSCAR: <https://oscar-project.github.io>
- OPUS: <https://opus.nlpl.eu/>

Dificultades del Web Crawling/Scrapping

- Algunas páginas bloquean accesos automatizados (CAPTCHAs, contenido bajo contraseña, sólo accesible a través de formularios, etc.).
- Cambios en la estructura HTML pueden romper el código de scraping.
- El contenido puede ser generado dinámicamente con JavaScript.

Contenido Generado con JavaScript

- Algunas páginas no cargan completamente en HTML estático.
- El contenido se genera dinámicamente mediante llamadas a APIs en JavaScript.
- Herramientas como requests no pueden acceder a este contenido directamente.

Headless Browsers

- Simulan un navegador real sin interfaz gráfica.
- Permiten ejecutar JavaScript y obtener el contenido final renderizado.
- Herramientas comunes:
 - ▶ Selenium
 - ▶ Requests-HTML
 - ▶ Puppeteer (para Node.js)

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

- La descarga de textos tiene implicaciones legales.
- El contenido en Internet suele estar protegido por restricciones de copyright.
- Es importante conocer las normativas antes de realizar estas prácticas.

- Convención de Berna para la Protección de Obras Literarias y Artísticas: si no se explicita el copyright, es el más restrictivo
- Leyes de protección de datos (GDPR en Europa, CCPA en EE.UU.).
- Leyes de derechos de autor (DMCA en EE.UU., Directiva de Copyright en la UE).
- Términos de servicio de los sitios web.

- No todo el contenido en línea es de dominio público.
- En EE.UU. existe el concepto de uso justo ("fair use") que permite usos determinados de material bajo copyright sin pedir permiso.
- Algunos casos de uso justo ("fair use") pueden justificar la recolección:
 - ▶ Investigación académica.
 - ▶ Análisis no comercial.
- Siempre verificar la licencia del contenido antes de usarlo.

Términos de Servicio de los Sitios Web

- Muchos sitios prohíben el scraping en sus términos de uso.
- Revisar el archivo `robots.txt` para conocer restricciones.
- Violaciones pueden llevar a acciones legales o bloqueos de IP.

Consejos para una Recolección Ética y Legal

- Respetar los términos de servicio y robots.txt.
- Solicitar permiso cuando sea posible.
- No sobrecargar servidores con solicitudes masivas.
- Anonimizar y proteger datos sensibles si se procesan datos personales.

El archivo robots.txt

- Archivo de texto ubicado en la raíz de un sitio web (p.e. `https://www.ua.es/robots.txt`).
- Define qué partes de la web pueden ser accedidas por bots.
- Ayuda a los administradores a gestionar el tráfico de scrapers y crawlers.
- Respetar este archivo es una buena práctica ética y técnica.

Ejemplo de robots.txt

```
User-agent: *  
Disallow: /admin/  
Disallow: /private/
```

- La regla anterior indica que los bots no deben acceder a /admin/ ni /private/.
- Algunas páginas permiten el acceso parcial a ciertas secciones.
- No todos los bots respetan robots.txt, pero los motores de búsqueda sí.

User-Agent

¿Qué es un User-Agent?

- Es una cadena de texto que identifica el software que realiza una solicitud a un servidor web.
- Se envía en la cabecera de cada petición HTTP.
- Los servidores pueden usar esta información para personalizar respuestas o restringir el acceso.

Ejemplo de User-Agent:

Googlebot (crawler de Google)

```
Mozilla/5.0 (compatible; Googlebot/2.1;  
+http://www.google.com/bot.html)
```

Consideraciones:

- Algunos sitios bloquean ciertos User-Agents.
- Se pueden personalizar en herramientas de scraping para evitar restricciones.

Tiempo de espera entre solicitudes

- Acceder repetidamente a un servidor sin pausas puede sobrecargarlo.
- Se recomienda establecer un tiempo de espera entre solicitudes (ej. 5 segundos).
- En Scrapy, se puede configurar con `DOWNLOAD_DELAY`.
- Respetar las limitaciones evita ser bloqueado y es una buena práctica.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

Preprocesamiento de texto: un paso esencial

- El preprocesamiento de texto prepara el texto en bruto para su procesamiento eficiente por algoritmos y modelos.
- Garantiza consistencia, reduce ruido y optimiza los datos para tareas posteriores.
- Diferentes tareas requieren diferentes pasos de preprocesamiento.

Algunas tareas frecuentes en el preprocesamiento de texto

- **Limpieza de formato:**

- ▶ Eliminar formatos no deseados, como etiquetas HTML o metadatos de PDF.

- **Tokenización del texto:**

- ▶ Dividir el texto en unidades más pequeñas como oraciones, palabras, subpalabras o caracteres.

- **Normalización del texto:**

- ▶ Normalizar puntuación y espacios.
- ▶ Convertir el texto a minúsculas para procesamiento sin distinción de mayúsculas.

- **Manejo de la puntuación:**

- ▶ Eliminar o conservar la puntuación según la aplicación.

- **Identificación de palabras vacías (stopwords):**

- ▶ Eliminar palabras comunes (e.g., *el*, *es*, *y*) para enfocarse en contenido significativo.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Por qué es necesario limpiar el formato?

- El texto en bruto a menudo viene en formatos no adecuados para modelos de PLN:
 - ▶ Archivos HTML con etiquetas y scripts.
 - ▶ Documentos PDF con metadatos e información de diseño.
 - ▶ Archivos Markdown con marcas de formato.
- En la mayoría de los casos, los datos relacionados con el formato agregan ruido al texto a procesar.

Eliminación de formato: HTML y PDF

- **HTML:**

- ▶ A menudo contiene etiquetas (`<div>`, `<script>`, etc.) y estilos.
- ▶ La extracción de contenido implica ignorar estos elementos.

- **PDF:**

- ▶ Puede incluir números de página, encabezados e imágenes.
- ▶ Las herramientas de extracción de texto pueden ayudar a recuperar solo el contenido textual.

Ejemplo: Eliminación de etiquetas HTML

Texto en bruto:

```
<html>
  <head><title>Ejemplo</title></head>
  <body>
    <h1>Hola , Mundo</h1>
    <p>Este es un texto de muestra.</p>
  </body>
</html>
```

Texto limpio:

Hola , Mundo
Este es un texto de muestra.

Técnicas para la eliminación de formato

- Expresiones regulares (Regex):
 - ▶ Usar patrones para identificar y eliminar elementos no deseados.
 - ▶ Ejemplo: Eliminar etiquetas HTML con el patrón `<.*?>`.
- Librerías y herramientas:
 - ▶ BeautifulSoup (Python): Para analizar y limpiar HTML.
 - ▶ PyPDF2 (Python): Para extraer texto de archivos PDF.
- Herramientas OCR:
 - ▶ Usar Reconocimiento Óptico de Caracteres para imágenes o texto escaneado.

¿Qué es BeautifulSoup?

- Es una librería de Python para extraer datos de páginas web.
- Permite navegar y buscar dentro del código HTML de una página.
- Se usa junto con `requests` para descargar el contenido web.

Ejemplo: Obtener el título de una página

Código en Python:

```
import requests
from bs4 import BeautifulSoup

url = "https://example.com"
response = requests.get(url)

soup = BeautifulSoup(response.text, "html.parser")

entradas = soup.find_all('div', {'class': 'documento'})

for i, entrada in enumerate(entradas):

    titulo = entrada.find('span', {'class': 'tituloLibro'}).
        getText()
```

¿Todo el contenido de una web es relevante?

Problema: Extraer el contenido principal de una página web eliminando elementos irrelevantes como menús, anuncios y enlaces repetitivos (boilerplate).

- Las páginas web contienen tanto contenido útil como elementos secundarios.
- Los motores de búsqueda y sistemas de análisis necesitan el texto limpio.
- Ejemplo: Noticias en línea, donde el artículo es el foco.

¿Qué es el Boilerplate?

- Elementos repetitivos en múltiples páginas.
- Menús de navegación, anuncios, pie de página.
- Ejemplo visual: Comparar una página de noticia con y sin boilerplate.

- **Basados en reglas:** Identificación de etiquetas HTML específicas.
- **Basados en aprendizaje automático:** Modelos entrenados para clasificar contenido relevante.
- **Algoritmos heurísticos:** Como Boilerpipe, que usa estructura del documento.

Herramientas Populares

- BeautifulSoup: Para analizar el HTML y filtrar elementos.
- Boilerpipe: Extracción heurística de contenido principal.
- Readability: Utilizado por navegadores para generar versiones limpias.

Usando Boilerpipe

```
from boilerpipe.extract import Extractor
url = "https://example.com/news"
extractor =
    Extractor(extractor='ArticleExtractor', url=url)
print(extractor.getText())
```

Extracción de texto de documentos PDF

- Los documentos PDF están diseñados para la presentación visual, no para la extracción de texto estructurado.
- Existen múltiples dificultades al intentar extraer texto de un PDF.
- Herramientas como `pdftotext`, `PyPDF2` y `pdfplumber` pueden ayudar, pero con limitaciones.

Dificultades en la Extracción de Texto de PDF

- **Codificación de caracteres:** Algunos PDF usan codificaciones especiales.
- **Texto embebido en imágenes:** OCR es necesario para extraer texto de imágenes.
- **Orden incorrecto:** El orden de lectura puede no coincidir con la disposición visual.
- **Fuentes incrustadas:** Algunas fuentes no estándar dificultan la extracción.

Ejemplo de Extracción con Python

Código con pdfplumber:

```
from pypdf import PdfReader

reader = PdfReader("example.pdf")
number_of_pages = len(reader.pages)
page = reader.pages[0]
text = page.extract_text()
```

Desafíos comunes en la eliminación de formato

- Manejo de datos ruidosos o incompletos.
- Conservación de una estructura significativa (e.g., tablas, párrafos).
- Gestión eficiente de archivos grandes o complejos.
- Formatos específicos del idioma (e.g., escritura de derecha a izquierda o caracteres especiales).

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - **Tokenización**
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Qué es la tokenización?

- Es el proceso de dividir el texto en unidades más pequeñas, llamadas **tokens**.
- Los tokens pueden representar:
 - ▶ Oraciones (también se llama a esta tarea *segmentación en oraciones*).
 - ▶ Palabras.
 - ▶ Caracteres.
 - ▶ Sub-palabras.
- Es un paso crítico para transformar texto en bruto en un formato adecuado para los algoritmos de PLN.

Niveles de tokenización

● Tokenización en oraciones:

- ▶ Divide el texto en oraciones.
- ▶ Ejemplo: *"El PLN es fascinante. La tokenización es esencial."*
- ▶ Tokens: [*"El PLN es fascinante."*, *"La tokenización es esencial."*]

● Tokenización en palabras:

- ▶ Divide el texto en palabras.
- ▶ Ejemplo: *"El PLN es fascinante"*
- ▶ Tokens: [*"El"*, *"PLN"*, *"es"*, *"fascinante"*]

● Tokenización en caracteres:

- ▶ Divide el texto en caracteres individuales.
- ▶ Ejemplo: *"PLN"*
- ▶ Tokens: [*"P"*, *"L"*, *"N"*]

● Tokenización en sub-palabras:

- ▶ Divide palabras en unidades potencialmente significativas.
- ▶ Ejemplo: *"increíble"*
- ▶ Tokens: [*"in"*, *"creí"*, *"ble"*]

Segmentación en oraciones

- Para la mayoría de los idiomas, se usa la **puntuación** (se divide por puntos, comas, etc.).
 - ▶ A veces es complicado: "*Este es el Dr. López. Él es el autor del blog saludparatodos.net.*"
- En algunos casos es posible usar el **formato**; por ejemplo, algunas etiquetas HTML delimitan un bloque de texto, como <p> o <h1>.
- **Algunos idiomas no usan puntuación** (por ejemplo, el tailandés).

ประวัติ

ภาษาไทยจัดอยู่ในกลุ่มภาษาไท (Tai languages) ภาษาหนึ่ง ซึ่งเป็นสาขาย่อยของตระกูลภาษาขร้า-ไท ภาษาไทยมีความสัมพันธ์อย่างใกล้ชิดกับภาษาในกลุ่มภาษาไทตะวันตกเฉียงใต้ภาษาอื่น ๆ เช่น ภาษาลาว ภาษาผู้ไท ภาษาคำเมือง ภาษาไทใหญ่ เป็นต้น รวมถึงภาษาตระกูลไทอื่น ๆ เช่น ภาษากูย ภาษาเขม่านาน ภาษาบูอี ภาษาไทลื้อ ที่พูดโดยชนพื้นเมืองบริเวณไทหนาน กวางสี กวางตุ้ง กุ้ยโจว ตลอดจนยูนนาน ไปจนถึงเวียดนามตอนเหนือ ซึ่งสันนิษฐานว่าจุดกำเนิดของภาษาไทยน่าจะมาจากบริเวณดังกล่าว

Estrategias de tokenización en palabras

- **Tokenización basada en espacios en blanco:**
 - ▶ No funciona bien con contracciones o puntuación.
 - ▶ Palabras separadas por guiones en inglés: *state-of-the-art*.
 - ▶ ¿Qué hacer con los idiomas que no usan espacios para separar palabras?
- **Uso de expresiones regulares:**
 - ▶ Permite identificar ciertos fenómenos: algunas contracciones en inglés, URLs, etc.
- **Tokenizadores específicos por idioma:** Adaptados para manejar características lingüísticas específicas.
 - ▶ Ejemplo: Tokenización en japonés con MeCab o SudachiPy.
 - ▶ Existen tokenizadores basados en conocimiento (diccionarios morfológicos) y otros basados en modelos estadísticos (por ejemplo, HMM).

¿Tokenización en sub-palabras?

Se ha vuelto muy popular en los modelos de PLN basados en redes neuronales:

- Aborda problemas con **palabras raras** y **palabras fuera de vocabulario** (OOV).
- Es eficiente para **idiomas morfológicamente ricos**.
- Mantiene un equilibrio entre la tokenización en palabras y en caracteres.

Enfoques para la tokenización en sub-palabras

La tarea se ha basado tradicionalmente en la **segmentación morfológica**.

Dos estrategias populares en la era neuronal:

- Codificación de pares de bytes (Byte Pair Encoding - BPE).
- Modelo de lenguaje de unigramas (Unigram Language Model).

Codificación de Pares de Bytes (BPE)

- Comienza dividiendo el texto en caracteres.
- Fusiona iterativamente los pares de caracteres o sub-palabras más frecuentes.
- Ejemplo:
 - ▶ Tokens iniciales: ['l', 'o', 'w', 'e', 'r']
 - ▶ Fusiona "l" y "o" → ['lo', 'w', 'e', 'r']
 - ▶ Fusiona "lo" y "w" → ['low', 'e', 'r']
- Beneficios:
 - ▶ Maneja palabras raras dividiéndolas en sub-unidades.
 - ▶ Tamaño compacto del vocabulario.

Modelo de Lenguaje Unigrama

- Pasos:

- ▶ Comienza con un gran vocabulario de sub-palabras potenciales (podría ser todas las sub-palabras posibles en el corpus).
- ▶ Asigna una probabilidad a cada una según su frecuencia observada en el corpus.
- ▶ Usa el vocabulario como un modelo unigrama que permite obtener la probabilidad de una palabra.
- ▶ Elimina iterativamente las sub-palabras que impacten mínimamente la probabilidad general del corpus.

- Beneficios:

- ▶ Permite múltiples segmentaciones con probabilidades.
- ▶ Más flexible que métodos deterministas como el BPE.

Comparación: BPE vs. Unigrama

- **BPE:**

- ▶ Determinista.
- ▶ Segmentación fija después del entrenamiento.

- **Unigrama:**

- ▶ Probabilístico.
- ▶ Permite múltiples segmentaciones válidas con probabilidades.

- **En común:**

- ▶ Ambos requieren pre-tokenización.
- ▶ Ambos permiten especificar el tamaño del vocabulario final.

- Se basa en Unigrama o BPE.
- Unigrama y BPE asumen que el corpus puede dividirse en palabras por espacios en blanco.
- SentencePiece omite esta suposición:
 - ▶ Incluye espacios en el vocabulario inicial de BPE.
 - ▶ Incluye sub-palabras que contienen espacios en el vocabulario inicial de Unigrama.
- Permite manejar idiomas que no usan espacios en blanco.
- Permite usar expresiones multi-palabra como elementos en el vocabulario final.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Qué es la Normalización de Texto?

- El proceso de convertir el texto en una forma estándar.
- Busca reducir la variabilidad del texto mientras se conserva el significado.
- Prepara el texto para un procesamiento consistente y efectivo en tareas de PLN.

Ejemplos:

- *"Hello,"* → *"hello"* (conversión a minúsculas).
- *"I've got 2 apples."* → *"i have two apples"* (normalización de contracciones y números).

Desafíos con Caracteres Unicode

- Texto moderno: generalmente se codifica con **Unicode** que soporta prácticamente todos los alfabetos.
- La amplia cobertura de Unicode puede provocar problemas.
- **Caracteres Visualmente Similares:**
 - ▶ `\á` (U+00E1) vs. `\á` (U+0061 + U+0301).
 - ▶ Parecen idénticos pero tienen representaciones subyacentes diferentes.
 - ▶ Con puntuación, el problema es aún mayor; revisa la puntuación UTF-8 en: <https://www.compart.com/en/unicode/category/Po>
- **Espacios No-Interrumpidos y Caracteres Invisibles:**
 - ▶ `\ "` (espacio no-interrumpido, U+00A0) vs. `\ "` (espacio, U+0020).
 - ▶ Introducen errores sutiles en el procesamiento.

Necesidades de Normalización Específicas para la Tarea

- La normalización de texto varía según la tarea de PLN.
- **Ejemplos:**
 - ▶ **Tareas sensibles a mayúsculas/minúsculas:**
 - ★ Reconocimiento de Entidades Nombradas (NER): Mantener las mayúsculas originales para identificar entidades como *"Apple"*.
 - ▶ **Eliminación de Puntuación:**
 - ★ Útil para modelos de bolsa de palabras.
 - ★ No siempre es adecuado para tareas como análisis de sentimiento.
 - ▶ **Eliminación de texto redundante:**
 - ★ Eliminar oraciones o párrafos duplicados o casi duplicados en un corpus largo.
 - ★ Útil cuando se tiene un corpus grande para entrenar modelos generativos.
- La normalización debe mantener un equilibrio entre generalidad y requisitos específicos de la tarea.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Qué son las Palabras Vacías?

Las palabras vacías son palabras comunes que se consideran de bajo valor semántico y se eliminan durante el preprocesamiento del texto para algunas tareas de PLN.

- Ejemplos: “el”, “es”, “en”, “sobre”, “a”, “y”, “para”
- Eliminarlas puede mejorar la eficiencia y centrarse en palabras más importantes.
- Son más comunes en idiomas con baja complejidad morfológica.

¿Cómo Detectar las Palabras Vacías?

Las palabras vacías pueden detectarse de varias formas:

- Listas predefinidas de palabras vacías (por ejemplo, NLTK, SpaCy).
- Enfoques basados en frecuencia:
 - ▶ Las palabras que aparecen muy frecuentemente en muchos documentos son posibles palabras vacías.
 - ▶ Las palabras comúnmente ocurrientes en los corpus son candidatas.
- Etiquetado de categorías léxicas:
 - ▶ Las palabras de función (por ejemplo, determinantes, preposiciones, conjunciones) se consideran a menudo palabras vacías.

Distribución de Frecuencia del Vocabulario

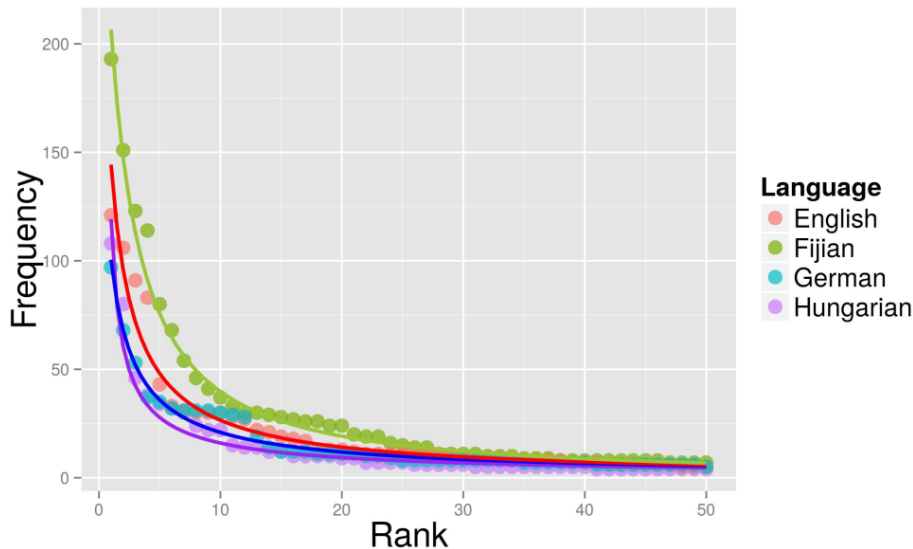


Figura 2 de: Bentz, C., Verkerk, A., Kiela, D., Hill, F., & Buttery, P. (2015).

Comunicación adaptativa: Los idiomas con más hablantes no nativos

La distribución Zipfiana del Vocabulario

- Cuando las palabras en un corpus se clasifican en orden decreciente de frecuencia, siguen una distribución zipfiana en la que:

$$\text{freq}(r) \propto \frac{1}{r}$$

- En otras palabras:
 - ▶ Unas pocas palabras en la mayoría de los idiomas tienen una frecuencia muy alta, y
 - ▶ la mayoría de las palabras en un idioma están en lo que se llama "la larga cola".
- Las palabras más frecuentes en un idioma suelen ser palabras funcionales (palabras vacías).

Implicaciones de Eliminar las Palabras Vacías en PLN

Eliminar las palabras vacías tiene varios efectos:

- **Enfoca en términos significativos:** Puede ayudar a enfatizar palabras que aportan contenido semántico para tareas como clasificación o agrupamiento.
- **Riesgo de perder contexto:** Eliminar demasiadas palabras vacías puede cambiar la estructura y el significado de la oración.
- **Consideraciones específicas de la tarea:** Algunas tareas (por ejemplo, análisis de sentimientos, modelado de lenguaje, etc.) pueden beneficiarse de retener palabras vacías.

¿Cómo debo preprocesar mi texto?

- Depende de la tarea:
 - ▶ Algunas decisiones son obvias (o al menos razonables) al conocer la tarea.
 - ▶ Otras decisiones requieren evaluación empírica.
- La tendencia actual de construir sobre modelos preentrenados hace que parte del preprocesamiento sea transparente.
- Un preprocesamiento adecuado tiene un gran impacto en el rendimiento de PLN posterior.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

Lingüística Computacional (LC)

- Un campo interdisciplinario en la intersección de:
 - ▶ Lingüística
 - ▶ Informática
- Se enfoca en modelar el lenguaje humano mediante métodos computacionales.
- Objetivos clave:
 - ▶ Analizar y comprender el lenguaje natural.
 - ▶ Desarrollar teorías lingüísticas respaldadas por herramientas computacionales.
- Temas de ejemplo:
 - ▶ Análisis de estructuras sintácticas.
 - ▶ Modelado de características morfológicas de una lengua.

Procesamiento de Lenguaje Natural (PLN)

- Un subcampo de la Inteligencia Artificial (IA) y el Aprendizaje Automático (ML).
- Se enfoca en diseñar algoritmos y sistemas para procesar datos de lenguaje natural.
- Objetivos clave:
 - ▶ Automatizar tareas basadas en lenguaje.
 - ▶ Permitir que las máquinas interactúen con los humanos a través del lenguaje.
- Aplicaciones de ejemplo:
 - ▶ Traducción automática.
 - ▶ Análisis de sentimientos.
 - ▶ Sistemas de respuesta a preguntas.

- **Lingüística Computacional (LC):**

- ▶ Enfatiza la comprensión teórica del lenguaje.
- ▶ Se basa en principios lingüísticos.

- **Procesamiento de Lenguaje Natural (PLN):**

- ▶ Se enfoca en las aplicaciones prácticas del procesamiento del lenguaje.
- ▶ Impulsado por la ingeniería y la eficiencia computacional.

Similitudes entre LC y PLN

- Ambos campos tratan con datos de lenguaje natural.
- Comparten métodos y herramientas, tales como:
 - ▶ Modelado de sintaxis y semántica.
 - ▶ Técnicas estadísticas y de aprendizaje automático.
- Trabajan para mejorar la interacción humano-computadora a través del lenguaje.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - **Análisis morfológico**
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Qué es el análisis morfológico?

- **Morfología:** Estudio de la estructura de las palabras.
- **Análisis Morfológico:** Descomposición de las palabras en:
 - ▶ **Raíces:** Componente semántico mínimo propio de las palabras.
 - ▶ **Morfemas:** Unidades más pequeñas de significado (raíces, prefijos, sufijos).
- Esencial para entender la formación de palabras, su significado y sus roles gramaticales.

¿Por qué es importante el análisis morfológico?

① Manejo de morfología rica:

- ▶ Idiomas como el finlandés, turco o árabe tienen estructuras de palabras complejas.

② Reducción de vocabulario:

- ▶ Agrupa formas flexionadas (por ejemplo, *run*, *ran*, *running*) en una sola forma base (**lema**).

③ Normalización de texto:

- ▶ Preprocesamiento para tareas como análisis de sentimientos y recuperación de información.

La mayoría de los idiomas en Europa tienen morfología bastante simple

Estamos acostumbrados a los **idiomas fusionales**: pocos morfemas flexivos que añaden información a una raíz.

Una palabra en inglés

Computed

Comput

ed

Raíz

sufijo indicando pasado

¿Cómo de compleja puede llegar a ser la morfología? 1/

También existen **idiomas aglutinantes**: combinan muchos morfemas flexivos, cada uno añadiendo nueva información.

Una palabra en finlandés

Taloissammekin

talo	i	ssa	mme	kin
casa	PLURAL	CASO INESSIVO	nuestro	también

¿Cómo de compleja puede llegar a ser la morfología? 2/

Y luego están los **idiomas polisintéticos**, que combinan muchas palabras en una sola.

Una palabra en inuktitut

annulaksikkanninginnajualugasulauqsimagumanngittsiaqgaluaqtunga

annulaksi	kkanni	nginna	jualu	gasu	lauqsima	guma	r
encarcelar	otra vez	realmente	mucho	intentar	siempre	querer	l

Nunca jamás querría intentar terminar en la cárcel otra vez, ni siquiera por un momento. (Johns, 2007)

¿Por qué es importante la morfología en la era de las tecnologías neuronales?

Dos usos principales:

- **En procesamiento del lenguaje natural (PLN):** principalmente útil para lenguas con pocos recursos
 - ▶ Simplifica el texto (ayuda a segmentar palabras).
 - ▶ Extrae información relevante para entender el significado.
 - ▶ Generación de texto morfológicamente correcto.
 - ▶ Apoyo a los estudiantes de segundas lenguas.
- **En lingüística computacional (LC):**
 - ▶ Anotación automática de corpus.
 - ▶ Apoya a los lingüistas en el descubrimiento de fenómenos lingüísticos.

Recursos relevantes para la morfología en PLN 1/

- **Unimorph:** Conjuntos de datos con listas exhaustivas de palabras en 169 idiomas, con tuplas que consisten en lemas, palabras de superficie, segmentación de palabras e información léxica (PoS, número, género, caso, etc.).
- **Universal Dependencies:** Corpora con (entre otra información) palabras anotadas con el lema, el PoS y información léxica adicional.

Proporciona anotaciones a nivel de tipo y es más exhaustivo (es más probable que cubra más palabras de un idioma).

lema	forma superficial	info. léx.
eat	eats	V;PRS;3;SG
eat	eating	V;V.PTCP;PRS
eat	ate	V;PST
eat	eaten	V;V.PTCP;PST
eat	eats	N;PL

Universal Dependencies

Proporciona **anotaciones a nivel de token** con tokens en un contexto.

Forma	Lema	PoS	info. léx.
He	he	PRON	PERS-P3SG-NOM Case=Nom...
ate	eat	VERB	PAST Mood=Ind—Tense=Past...
a	a	DET	IND-SG Definite=Ind...
mouthful	mouthful	NOUN	SG-NOM Number=Sing

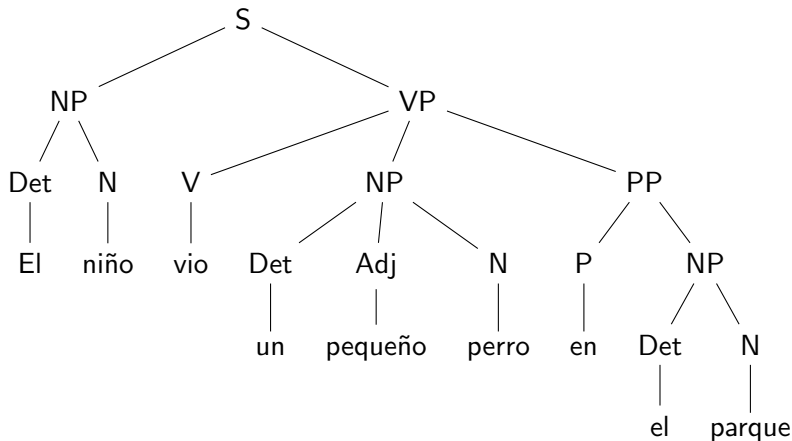
- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - **Análisis sintáctico**
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

¿Qué es el análisis sintáctico?

- El análisis sintáctico tiene como objetivo determinar la estructura de una oración.
- Proporciona representaciones que ayudan a entender las relaciones entre las palabras.
- Dos estrategias principales:
 - ▶ Estructura de frases (Gramática de constituyentes)
 - ▶ Estructura de dependencia (Gramática de dependencias)

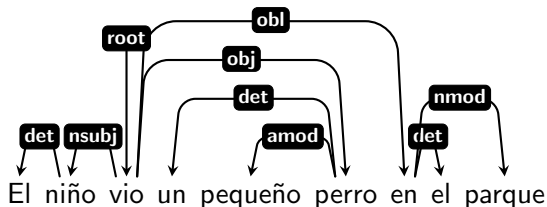
Estructura de frases

- Representa la sintaxis como frases anidadas.
- Comúnmente asociada con la gramática de constituyentes.



Estructura de dependencia

- Representa la sintaxis como relaciones dirigidas entre las palabras.
- Captura las dependencias directamente.



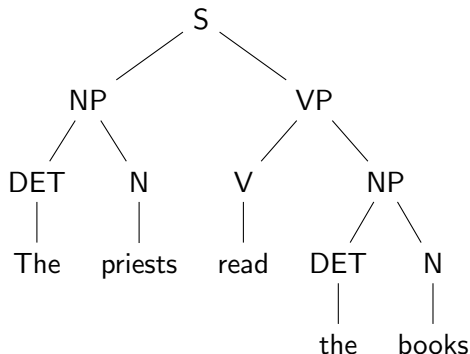
Por qué la estructura de dependencia es más universal

- La gramática de dependencias se centra en las relaciones palabra a palabra.
- Es más fácil de aplicar a idiomas con diferentes órdenes de palabras (por ejemplo, SVO, OVS, SOV, etc.).

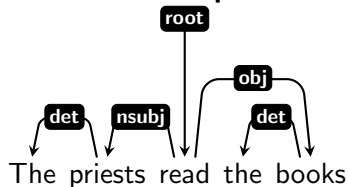
Ejemplo: Análisis sintáctico en dos idiomas 1/

Inglés (SVO): The priests read the books.

Estructura de frases:



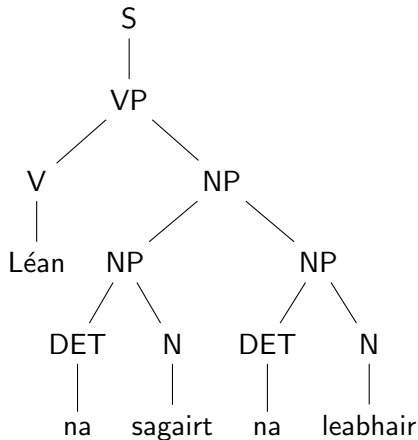
Estructura de dependencia:



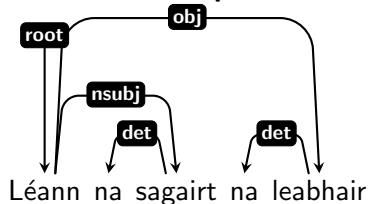
Ejemplo: Análisis sintáctico en dos idiomas 2/

Irlandés (VSO): Léann na sagairt na leabhair.

Estructura de frases:



Estructura de dependencia:



¿Por qué es importante la sintaxis en la era de las tecnologías neuronales?

Dos usos principales:

- **En PLN:** útil principalmente para lenguas con pocos recursos
 - ▶ Divide el texto en fragmentos significativos.
 - ▶ Desambiguación del texto.
 - ▶ Modelos mejorados con conocimiento (resumen, traducción, etc.).
 - ▶ Ayuda a identificar entidades nombradas.
- **En LC:**
 - ▶ Anotación automática de corpus.
 - ▶ Búsqueda de estructuras sintácticas específicas en corpus.
 - ▶ Apoyo para lingüistas en descubrimiento de fenómenos lingüísticos.

Universal Dependencies

Anotación tanto de **información morfológica** como de **dependencias sintácticas**.

Forma	Lema	PoS	info. léx.	Dep.	Tipo de dep.
You	you	PRON	PERS-P2	3	nsubj
can	can	AUX	PRES-AUX VerbForm=Fin	3	aux
change	change	VERB	INF VerbForm=Inf	0	root
the	the	DET	DEF Definite=Def	6	det
security	security	NOUN	SG-NOM Number=Sing	6	compound
mode	mode	NOUN	SG-NOM Number=Sing	3	obj
...	...	...	...	...	...

¿Qué herramientas se pueden usar para el análisis de dependencias?

Muchas herramientas. Algunas de las más populares:

- **Stanza:**

- ▶ Análisis basado en grafos implementado como una red neuronal.
- ▶ Multilingüe desde su creación.
- ▶ Más exhaustivo, pero también más lento.

- **ScyPy:**

- ▶ Implementación estadística de modelos de transición.
- ▶ Inicialmente enfocado al inglés, ahora cubre una amplia gama de idiomas.
- ▶ Más propenso a cometer errores con fenómenos lingüísticos de largo alcance, pero más rápido.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
- **Antonimia:** las palabras son opuestas
- **Similitud:** las palabras comparten algunos aspectos de su significado
- **Relación:** las palabras pertenecen al mismo *campo semántico*
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
 - ▶ *big* y *large*
 - ▶ sí, pero: *my big sister* $\neq$ *my large sister*
 - ▶ **Principio lingüístico del contraste:** forma diferente → significado diferente
- **Antonimia:** las palabras son opuestas
- **Similitud:** las palabras comparten algunos aspectos de su significado
- **Relación:** las palabras pertenecen al mismo *campo semántico*
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
- **Antonimia:** las palabras son opuestas
 - ▶ *big* vs. *small*
 - ▶ *up* vs. *down*
- **Similitud:** las palabras comparten algunos aspectos de su significado
- **Relación:** las palabras pertenecen al mismo *campo semántico*
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
- **Antonimia:** las palabras son opuestas
- **Similitud:** las palabras comparten algunos aspectos de su significado
 - ▶ *cow, horse* → rumiantes, tamaño, etc.
 - ▶ *pen, pencil* → forma, propósito, etc.
- **Relación:** las palabras pertenecen al mismo *campo semántico*
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
- **Antonimia:** las palabras son opuestas
- **Similitud:** las palabras comparten algunos aspectos de su significado
- **Relación:** las palabras pertenecen al mismo *campo semántico*
 - ▶ *coffee, cup* → el café se sirve en tazas.
 - ▶ *car, wheel* → los coches tienen cuatro ruedas.
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)

Las palabras están relacionadas por su significado

- **Sinonimia:** las palabras significan (casi) lo mismo
- **Antonimia:** las palabras son opuestas
- **Similitud:** las palabras comparten algunos aspectos de su significado
- **Relación:** las palabras pertenecen al mismo *campo semántico*
- **Connotación:** connotaciones independientes del significado (por ejemplo, sentimiento)
 - ▶ *reproducir* vs. *plagiar*.
 - ▶ *mayores* vs. *ancianos*.

Introducción a la Semántica Vectorial

- **Semántica vectorial:** Un método de representar los significados de las palabras mediante vectores en un espacio de n dimensiones.
- Las palabras se representan como puntos en este espacio, donde:
 - ▶ La distancia entre los vectores refleja la similitud semántica de las palabras.
 - ▶ Las palabras que aparecen en contextos similares están más cerca en el espacio vectorial.
- **Idea central:** *“Conocerás una palabra por la compañía que tiene”* (Firth, 1957).

Ejemplo: Palabras en el mismo contexto

Supongamos que ves estas oraciones:

- El **ong choi** es delicioso **salteado con ajo**.
- El **ong choi** está delicioso **con arroz**.
- **Hojas** de **ong choi** con **salsas saladas**.

Y también has visto estas:

- ... arroz con **espinacas** salteadas con ajo.
- Los tallos y las hojas de las **acelgas** están deliciosos.
- **Berzas** y otras verduras de hojas saladas.

Conclusión:

- Seguramente **Ong choi** estará cerca de palabras como **espinacas**, las **acelgas** o las **berzas** en el espacio n -dimensional.
- Aparece en contextos parecidos, acompañando a palabras como: **"hojas"**, **"deliciosos"** y **"saladas"**.

Introducción a los Embeddings

- **Embeddings:**

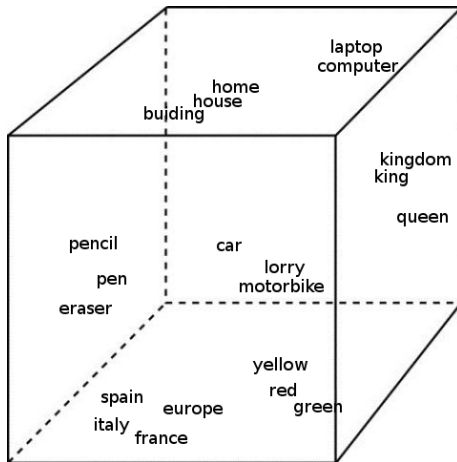
- ▶ Representaciones vectoriales densas de palabras en un espacio vectorial continuo.
- ▶ Capturan relaciones semánticas y sintácticas entre las palabras.
- Muchas estrategias existentes: Word2Vec, GloVe, Sent2Vec, BERT, etc.
- Generalmente se aprenden con redes neuronales.

¿Por qué Redes Neuronales para Embeddings?

- Las redes neuronales son excelentes para aprender embeddings porque:
 - ▶ Pueden aprender relaciones complejas y no lineales a partir de grandes cantidades de datos.
 - ▶ Las representaciones se ajustan para optimizar el rendimiento en tareas posteriores (por ejemplo, clasificación, traducción).
- Idea clave:
Los embeddings se aprenden durante el proceso de entrenamiento de una red neuronal en una tarea.
- Ejemplo:
 - ▶ En una tarea de análisis de sentimientos, los embeddings capturan matices semánticos relevantes para predecir el sentimiento.

Visualización de Embeddings de Palabras

- Las palabras semánticamente relacionadas (por ejemplo, "lápiz", "bolígrafo", "borrador") se agrupan.
- Las palabras con significados independientes están más separadas.



Principio de la composicionalidad semántica

El significado de una **expresión compleja** (una oración) está determinado por los **significados de sus partes constituyentes** (palabras) y **la forma en que se combinan** (sintaxis).

- **Lexema:** Una unidad de significado en el lenguaje, independiente de las formas flexivas.
 - ▶ Ejemplo: El lexema *run* cubre *runs*, *running*, *ran*.
- **Sentido de la palabra:** El significado específico de una palabra en un contexto dado.
 - ▶ Ejemplo: *bank* (institución financiera) vs. *bank* (orilla del río).

Introducción a la Representación de Documentos

- Representar el texto numéricamente es esencial para los modelos de aprendizaje automático.
- **Bolsa de Palabras (BoW):**
 - ▶ Representa los documentos como un vector de recuentos de palabras.
 - ▶ Ignora la gramática, el orden de las palabras y el contexto.
- **Frecuencia de Término - Frecuencia Inversa de Documento (TF-IDF):**
 - ▶ Asigna pesos a las palabras según su importancia en el documento y en el corpus.
 - ▶ Reduce el impacto de palabras frecuentes pero poco informativas (por ejemplo, “el”, “la”).

Creación de un Vector de Bolsa de Palabras (BoW)

- Dado un corpus:

Documento 1: "The cat sat on the mat."

Documento 2: "The dog lay on the rug."

- Vocabulario:

{cat, sat, mat, dog, lay, rug, on, the}

- Representa cada documento como un vector de recuentos de palabras:

Documento 1: [1, 1, 1, 0, 0, 0, 1, 2]

Documento 2: [0, 0, 0, 1, 1, 1, 1, 2]

- Filas = documentos, columnas = recuentos de palabras.

Creación de un Vector TF-IDF

- **Paso 1: Frecuencia de Término (TF):**

$$TF = \frac{\text{Número de ocurrencias del término en el documento}}{\text{Total de términos en el documento}}$$

- **Paso 2: Frecuencia Inversa de Documento (IDF):**

$$IDF = \log \frac{\text{Número total de documentos}}{\text{Número de documentos que contienen el término}}$$

- **Paso 3: Pesos TF-IDF:**

$$TF\text{-}IDF = TF \times IDF$$

- Ejemplo con 100 documentos:

- ▶ Palabra: "el" (aparece en todos los documentos).
- ▶ $IDF = \log \frac{100}{100} = 0$ (baja importancia).
- ▶ Palabra: "gato" (aparece en un solo documento).
- ▶ $IDF = \log \frac{100}{1} = \log 100 = 2$ (mayor importancia).

Limitaciones:

- Ignoran el contexto y el orden de las palabras.
- BoW da la misma relevancia a todas las palabras.
- TF-IDF puede asignar pesos bajos a palabras semánticamente importantes.
- Ambos métodos producen vectores dispersos para grandes vocabularios (vectores grandes con vocabulario amplio).

Introducción a los Embeddings de Oraciones

- **Embeddings de oraciones:** embeddings que capturan la semántica de toda una oración.
- A diferencia de BoW o TF-IDF:
 - ▶ Los embeddings son **densos** (baja dimensión) en lugar de dispersos.
 - ▶ Se aprenden automáticamente a partir de los datos en lugar de basarse en simples conteos o ponderaciones.
- Diferentes modelos dependiendo de su uso
 - ▶ **Embeddings de oraciones de propósito general:** USE, SBERT, SimCSE.
 - ▶ **Tareas multilingües:** LASER, Multilingual USE, XLM-RoBERTa.
 - ▶ **Opciones livianas:** MiniLM, DistilBERT.
 - ▶ **Embeddings de párrafos:** Doc2Vec, modelos basados en GPT, T5.

Distancia entre representaciones vectoriales

- Para comparar dos representaciones vectoriales (por ejemplo, para oraciones o documentos), calculamos su **distancia** o **similitud**.
- Medidas comunes:
 - ▶ **Distancia euclídea**: Mide la distancia en línea recta entre los vectores.
 - ▶ **Similitud del coseno**: Mide el coseno del ángulo entre los vectores en el espacio vectorial.
- ¿Por qué usar similitud del coseno?
 - ▶ Se enfoca en la orientación, no en la magnitud.
 - ▶ Ideal para búsqueda de similitud entre textos (no nos importa la magnitud, sino la relación entre palabras).

Cálculo de la Similitud del Coseno

- **Fórmula para la similitud del coseno:**

$$\text{similitud del coseno} = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|}$$

Donde:

- ▶ $\vec{u} \cdot \vec{v}$ = producto escalar de los dos vectores.
- ▶ $\|\vec{u}\|$ y $\|\vec{v}\|$ = magnitudes (normas) de los vectores.

- **Pasos:**

- 1 Calcular el producto escalar: $\vec{u} \cdot \vec{v} = \sum_{i=1}^n u_i v_i$
- 2 Calcular las magnitudes: $\|\vec{u}\| = \sqrt{\sum_{i=1}^n u_i^2}$, $\|\vec{v}\| = \sqrt{\sum_{i=1}^n v_i^2}$
- 3 Dividir el producto escalar por el producto de las magnitudes.

- **Resultado:**

- ▶ El valor varía de -1 (opuesto) a +1 (idéntico).
- ▶ Un valor más alto significa mayor similitud.

- 1 Web Scrapping y Web Crawling
 - Buenas prácticas de crawling/scraping
- 2 Preprocesamiento de texto
 - Eliminación de formato
 - Tokenización
 - Normalización de Texto
 - Palabras Vacías (Stopwords)
- 3 La lingüística computacional y el procesamiento del lenguaje natural
 - Análisis morfológico
 - Análisis sintáctico
 - Representación de la semántica
 - La semántica a nivel de palabras
 - La semántica a nivel de textos
 - Resumen

- Es muy relevante en LC, ya que permite el análisis a nivel de palabra de los corpus y ayuda a entender e identificar fenómenos lingüísticos.
- Especialmente relevante para lenguas con pocos recursos, para los cuales todavía juega un papel importante en el PLN.
- Útil para lidiar con morfología rica, reducir el tamaño del vocabulario y mejorar los modelos de PLN.
- Unimorph y Universal Dependencies son dos de los recursos multilingües más relevantes para esta tarea.

- Está dirigido a entender la estructura y las relaciones entre las palabras en una oración.
- Tanto la estructura de sintagmas como la estructura de dependencias proporcionan información valiosa, siendo la estructura de dependencias más flexible entre lenguas.
- La sintaxis sigue siendo importante en PLN, especialmente para lenguas con pocos recursos, ayudando en tareas como desambiguación, resumen automático y reconocimiento de entidades nombradas.
- El uso de dependencias universales y herramientas de análisis como Stanza y ScyPy mejora el análisis sintáctico en un contexto multilingüe.

Representaciones Vectoriales del Texto

- Las palabras tienen relaciones complejas basadas en su significado: sinonimia, antonimia, similitud, relación, etc.
- El contexto de las palabras ayudan a identificar relaciones entre palabras.
- La semántica vectorial permite representar los significados de las palabras como vectores numéricos en espacios de alta dimensión.
- Métodos como **BoW** y **TF-IDF** proporcionan representaciones simples pero potentes para el texto en determinados escenarios.
- El uso de *embeddings* permite capturar relaciones semánticas más sofisticadas, y obtener representaciones más compactas y eficientes.
- Comparar representaciones vectoriales con métricas como la **similitud del coseno**, permite la comparación eficiente de textos y apoya una amplia gama de aplicaciones de PLN.